Admin 楼层结构标注页设计规范

> 版本：v1.0
>
> 配套文档：
>
> - `FLOOR_MAP_HYBRID_RENDERING_PLAN.md`（架构）
> - `FLOOR_MAP_EXTRACTOR_SPEC.md`（抽取算法）
> - `FLOOR_MAP_API_SPEC.md`（API 契约）
> - `schemas/floor_map.v2.schema.json`（JSON Schema）
> - 视觉参照：`frontend/src/app/pages/FloorPlan.tsx`（手绘风原型）

0. 目标与范围

为运营/资产管理员提供**单楼层结构语义审核界面**，把 DXF 自动抽取出的候选结构（电梯/楼梯/卫生间/核心筒等）以图形方式呈现，允许人工**确认 / 修正 / 增删 / 拖动**后写回后端，最终决定该楼层使用 `vector` 还是 `semantic` 渲染模式。

不做：完整 CAD 编辑器、单元（房间）切分（已由 `split_dxf_by_floor.py` 完成）、跨楼层批量编辑（v1.0 单楼层串行处理）。

1. 路由与权限

项	值
路径	`/assets/buildings/:buildingId/floors/:floorId/structures`
菜单挂靠	资产管理 → 楼栋详情 → 楼层卡片右上角「结构标注」按钮
角色	`assets.write` 可编辑；`assets.read` 仅查看（按钮变灰，PUT 调用前端拒绝）
路由守卫	复用 `admin/src/router/guards.ts` 已有逻辑

2. 目录结构

```
admin/src/views/assets/floor-structures/
  FloorStructureAnnotatorView.vue      # 容器页（≤ 250 行；含状态分发，无业务逻辑）
  components/
    AnnotatorToolbar.vue               # 顶部工具条：模式切换、保存、撤销、重抽取
    AnnotatorCanvas.vue                # SVG 画布主体，承载 outline + structures + units
    StructureLayer.vue                 # 渲染所有 structures，含选中态/拖拽
    UnitOverlayLayer.vue               # 渲染只读 units 多边形（参考 FloorPlan.tsx 的 RoomOverlay）
    StructureInspector.vue             # 右侧属性面板：编辑 type/label/code/gender
    CandidatePalette.vue               # 左侧候选清单：未确认结构列表，点击→ 加入画布
    RenderModeSwitch.vue               # 提交时调用 PATCH /render-mode 的开关
```

```
composables/
  useFloorMapDraft.ts                # 草稿状态管理（包装 store action）
  useCanvasInteraction.ts            # 拖拽/缩放/键盘快捷键
__tests__/
  FloorStructureAnnotatorView.spec.ts
  structureStore.spec.ts
```

> 单文件硬上限沿用工作区规范：`*View.vue` ≤ 250 行、`*.vue` 组件 ≤ 250 行、`*Store.ts` ≤ 200 行。

3. 状态管理（Pinia）

3.1 Store：`useFloorStructureStore`

文件：`admin/src/stores/floorStructure.ts`（setup 风格）。

字段	类型	说明
`floorId`	`Ref<string \	null>`
`viewport`	`Ref<Viewport \	null>`
`outline`	`Ref<Outline \	null>`
`candidates`	`Ref<Structure[]>`	自动抽取的候选（`source: auto`）
`structures`	`Ref<Structure[]>`	当前画布上已纳入的结构（含 manual）
`units`	`Ref<Unit[]>`	只读单元列表（用于参考）
`renderMode`	`Ref<'vector' \	'semantic'>`
`selectedId`	`Ref<string \	null>`
`loading`	`Ref<boolean>`	加载/保存中
`error`	`Ref<string \	null>`
`dirty`	`Computed<boolean>`	草稿与初始 `structures` 是否不同

actions（按工作区错误处理规则，全部 `try/catch` 包装为 `error.value = e instanceof ApiError ? e.message : '操作失败'`）：

action	调用	说明
`loadCandidates(floorId)`	`GET /api/floors/:floorId/structures/candidates`	初始化所有 ref
`addFromCandidate(id)`	本地	从 candidates → structures，标记 `source: 'auto'`
`removeStructure(id)`	本地	从 structures 移除
`updateStructure(id, patch)`	本地	改 type/label/rect 等

`addManualStructure(rect)`	本地	用户拖框新建，`source: 'manual'`
`save()`	`PUT /api/floors/:floorId/structures`	提交 structures + outline + windows
`switchRenderMode(mode)`	`PATCH /api/floors/:floorId/render-mode`	单独操作，不与 save 合并
`reset()`	本地	丢弃草稿、重新装载

3.2 类型定义

`admin/src/types/floorStructure.ts` 复用后端字段命名，直接对应 `floor_map.v2.schema.json` `$defs`，避免重复定义。

```
export type StructureType = 'core' | 'elevator' | 'stair' | 'restroom' | 'equipment' |
'corridor';
export interface Rect { x: number; y: number; w: number; h: number }
export interface Structure {
  // 前端临时 ID，仅用于列表渲染，不提交后端
  _localId: string;
  type: StructureType;
  rect: Rect;
  label?: string | null;
  code?: string | null;
  gender?: 'M' | 'F' | 'unknown' | null;
  source: 'auto' | 'manual';
  confidence?: number;
}
```

4. 交互流程

4.1 进入页面

1. 路由解析 `floorId`
2. `loadCandidates` → 后端返回 `outline / structures / windows / units / viewport`
3. 画布渲染：
   - 底层：`outline` 灰色描边
   - 中层：`units` 半透明多边形（只读，仅用于对位）
   - 上层：`structures` 彩色填充 + 选中描边

4.2 编辑动作

操作	触发	副作用
点击候选项	`CandidatePalette` 列表项	`addFromCandidate`
选中画布结构	单击 SVG `<g>`	`selectedId = id`
拖动结构	鼠标按下并移动	`updateStructure(id, { rect })`
修改属性	`StructureInspector` 表单	`updateStructure(id, patch)`
删除	`Delete` 键 / 工具条按钮	`removeStructure`
框选新建	工具条「+ 矩形」后画布拖拽	`addManualStructure`
撤销	`Ctrl/Cmd + Z`	草稿历史栈回退（v1.0 仅支持单步撤销）
保存	工具条「保存」	`save()` → 成功后刷新 ETag，`dirty = false`
切换 vector/semantic	`RenderModeSwitch`	`switchRenderMode`，与保存解耦

4.3 离开守卫

- 路由 `beforeRouteLeave`：若 `dirty === true`，弹 `ElMessageBox.confirm` 提示「有未保存的修改，确认离开？」
- 复用 `admin/src/router/guards.ts` 的 `unsavedChangesGuard`（若不存在则新增）

5. 画布渲染细节

`AnnotatorCanvas.vue` 关键约定：

项	值
根元素	`<svg :viewBox="` 0 0 ${viewport.width} ${viewport.height} `">`
坐标系	与后端 JSON 完全一致（已经过 Y 翻转，左上为原点）
缩放	CSS `transform: scale()` 包外层 div；不修改 viewBox
平移	拖拽空白区域；按住空格键启用
网格	100×100 单位浅色网格（可关）便于对位
颜色	类型 → token 映射（禁止硬编码）

颜色 token 在 `admin/src/constants/ui_constants.ts` 增加：

```
export const STRUCTURE_TYPE_COLORS: Record<StructureType, string> = {
  core:       'var(--el-color-info-light-3)',
  elevator:   'var(--el-color-primary-light-3)',
  stair:      'var(--el-color-success-light-3)',
  restroom:   'var(--el-color-warning-light-3)',
```

```
  equipment: 'var(--el-color-info-light-5)',
  corridor:  'var(--el-color-info-light-7)',
};
```

> 严格遵守工作区状态色语义：`auto` 候选用低饱和度，`manual` 边框加粗 1px 提示「人工新增」。

6. 表单与校验

`StructureInspector.vue` 字段约束直接由 `floor_map.v2.schema.json` 派生：

字段	显示	校验
`type`	下拉单选	必填，6 种枚举
`label`	输入框	选填，maxLength 32
`code`	输入框	`type === 'elevator'` 时必填，正则 `^[A-Z]\d{1,3}$`
`gender`	Radio	`type === 'restroom'` 时必填，男/女/未知
`rect.w/h`	数字输入	最小 1，最大不超过 viewport
`confidence`	只读 Tag	仅 `auto` 显示，百分比

校验失败时禁用「保存」按钮并在表单字段下方显示错误。

7. API 调用顺序

时机	顺序	备注
进入页面	`GET /candidates`	一次性加载所有数据
保存结构	`PUT /structures`	带 `If-Match: <etag>` 防并发
切换模式	`PATCH /render-mode`	独立操作

任意请求返回 `412 Precondition Failed` 时，提示「数据已被他人更新，请刷新」并自动调用 `loadCandidates` 重置。

8. 测试清单

`admin/e2e/floor-structures.test.ts`（Playwright）：

1. 加载页面 → 候选清单非空 → 自动渲染候选结构
2. 添加候选 → 修改属性 → 保存 → 列表中 `source: auto` 正常
3. 框选新建矩形 → 设为 `restroom` + `gender: F` → 保存
4. 切换模式 `vector → semantic` → 楼层卡片 `render_mode` 更新
5. 离开守卫：有未保存改动 → 弹窗 → 取消停留

`admin/src/stores/__tests__/floorStructure.spec.ts`（Vitest）：

1. `addFromCandidate` 后 `dirty === true`
2. `save` 失败时 `error` 被赋值
3. `reset` 恢复到初始 structures

9. 实施分期

阶段	范围
P1	Store + GET/PUT 联调 + 候选展示 + 保存按钮（无拖拽）
P2	拖拽、属性面板、撤销
P3	框选新建 + render mode 切换 + 离开守卫
P4	E2E 测试 + 视觉打磨（与 FloorPlan.tsx 对齐线条粗细/圆角）

10. 与既有代码的对齐

既有	复用方式
`frontend/src/app/pages/FloorPlan.tsx`	仅作视觉参照（手绘风线条粗细 1.5px、墙体灰阶、单元圆角 4px）
`admin/src/api/client.ts`	HTTP 通过现有 axios 封装；新增 `admin/src/api/modules/floorStructures.ts` 三个函数
`admin/src/constants/api_paths.ts`	增加三条路径常量，禁止组件内硬编码
`admin/src/views/assets/` 已有楼层卡片	增「结构标注」按钮 + 显示当前 `render_mode` 标签

Admin 楼层结构标注器 — 实现规范

> 关联：FLOOR_MAP_HYBRID_RENDERING_PLAN.md、FLOOR_MAP_API_SPEC.md、floor_map.v2.schema.json
>
> 使用框架：Vue 3 + TypeScript + Element Plus + Pinia + SVG 原生绘制（不引入第三方 canvas 库）
>
> 遵循约束：vue-admin.instructions.md、copilot-instructions.md 中 Admin 分层规则

---

0. 功能定位

供资产管理员对单个楼层进行结构标注：在自动抽取候选项基础上，编辑/新增/删除核心筒、电梯、楼梯、卫生间、立柱、窗户位置等，最终保存为 `floor_map.json` v2，并切换该楼层渲染模式为 `semantic`。

非目标：不替代 hotzone 房间标注（已存在），不处理 DXF 上传（属另一流程）。

---

1. 路由与权限

1.1 路由

```
// admin/src/router/index.ts 增补
{
  path: '/assets/buildings/:buildingId/floors/:floorId/structures',
  name: 'FloorStructureAnnotator',
  component: () => import('@/views/assets/floor-structures/AnnotatorView.vue'),
  meta: { requiresAuth: true, permission: 'assets.write' }
}
```

1.2 入口

`AssetsListView` 楼层卡片新增按钮 「**结构标注**」，仅当用户拥有 `assets.write` 权限时显示。

1.3 权限校验

`router.beforeEach` 守卫已根据 `meta.permission` 阻拦；后端 PUT/PATCH 端点二次校验。

---

2. 目录结构

```
admin/src/
  views/assets/floor-structures/
    AnnotatorView.vue                # 主页面 (≤ 250 行)
    components/
      CandidatesPanel.vue            # 左侧候选项列表
      CanvasStage.vue                # 中央 SVG 画布
      InspectorPanel.vue             # 右侧属性编辑
      Toolbar.vue                    # 顶部工具栏
      RenderModeSwitch.vue           # 切换 vector/semantic
  api/modules/floorStructures.ts     # 4 个 API 函数
  stores/floorStructuresStore.ts     # Pinia store
  types/floorMap.ts                  # FloorMap v2 TypeScript 类型 (从 schema 派生)
  constants/api_paths.ts             # 增加 4 条路径常量
```

文件复杂度遵循 copilot-instructions.md "文件复杂度超限时的拆分策略"：单 Vue 文件 ≤ 250 行，store ≤ 200 行。

---

3. API 路径常量

```
// admin/src/constants/api_paths.ts (增补)
```

```
export const API_PATHS = {
  // ...
  floorStructureCandidates: (floorId: string) =>
    `/api/floors/${floorId}/structures/candidates`,
  floorStructures: (floorId: string) => `/api/floors/${floorId}/structures`,
  floorRenderMode: (floorId: string) => `/api/floors/${floorId}/render-mode`,
};
```

---

4. TypeScript 类型

```
// admin/src/types/floorMap.ts
export type StructureType =
  | 'core' | 'elevator' | 'stair' | 'restroom'
  | 'shaft' | 'column' | 'corridor' | 'lobby';

export interface Rect { x: number; y: number; w: number; h: number; }

export interface Structure {
  type: StructureType;
  rect: Rect;
  label?: string;
  code?: string;
  gender?: 'M' | 'F' | 'U';
  source: 'auto' | 'manual';
}

export interface WindowSegment {
  side: 'N' | 'S' | 'E' | 'W';
  x: number;
  width: number;
  source?: 'auto' | 'manual';
}

export interface Outline {
  type: 'rect' | 'polygon';
  // 当 type=rect
  x?: number; y?: number; w?: number; h?: number;
  // 当 type=polygon
  points?: [number, number][];
}

export interface FloorMapV2 {
  schema_version: '2.0';
  viewport?: { width: number; height: number };
  outline: Outline;
  structures: Structure[];
  windows?: WindowSegment[];
  north?: { x: number; y: number; rotation_deg?: number };
  units?: unknown[]; // 只读拼接
}
```

> 类型字段对齐 floor_map.v2.schema.json。修改 schema 后必须同步更新这里。

---

5. Pinia Store

```
// admin/src/stores/floorStructuresStore.ts
export const useFloorStructuresStore = defineStore('floorStructures', () => {
  const candidates = ref<FloorMapV2 | null>(null);   // GET candidates 返回值
  const draft = ref<FloorMapV2 | null>(null);         // 编辑中草稿(浅拷贝候选项)
  const renderMode = ref<'vector' | 'semantic'>('vector');
  const loading = ref(false);
  const saving = ref(false);
  const error = ref<string | null>(null);
  const dirty = ref(false);

  // 历史栈支持撤销/重做
  const history = ref<FloorMapV2[]>([]);
  const historyIndex = ref(-1);

  // actions
  async function loadCandidates(floorId: string) { /* ... */ }
  async function loadConfirmed(floorId: string) { /* ... */ }
  async function save(floorId: string) { /* PUT /structures */ }
  async function setRenderMode(floorId: string, mode: 'vector' | 'semantic') { /* ... */ }
  function addStructure(s: Structure) { /* push + 历史 */ }
  function updateStructure(idx: number, patch: Partial<Structure>) {}
  function removeStructure(idx: number) {}
  function undo() {}
  function redo() {}

  return {
    candidates, draft, renderMode, loading, saving, error, dirty,
    loadCandidates, loadConfirmed, save, setRenderMode,
    addStructure, updateStructure, removeStructure, undo, redo,
  };
});
```

错误处理统一：`catch (e) { error.value = e instanceof ApiError ? e.message : '操作失败,请重试'; }`（copilot-instructions.md）。

---

6. 主视图布局

```
┌──────────────────────────────────────────────────────────────────────┐
│ Toolbar  [新增矩形] [删除] [撤销] [重做] [预览] [保存] [模式]│
├──────────┬──────────────────────────────────────────┬──────────────┤
│          │                                          │              │
│ 候选项列表│              SVG 画布                    │  属性面板 │
```

```
│ (左 240) │        (居中,可缩放/拖动)              │  (右 320)  │
│          │                                        │            │
│ • 核心筒 │   ┌──────────────────┐                 │ type:[v]   │
│ • 电梯E1 │   │  outline + 半透明 │                 │ x: [   ]   │
│ • 卫M    │   │ 结构矩形 + 选中态 │                 │ y: [   ]   │
│ ...      │   │  + 窗户橙线      │                 │ w: [   ]   │
│          │   └──────────────────┘                 │ h: [   ]   │
│          │                                        │ label:[ ]  │
└──────────┴────────────────────────────────────────┴────────────┘
```

6.1 三栏组件

- `CandidatesPanel.vue`：列出 `draft.structures`，支持点击选中、勾选可见性
- `CanvasStage.vue`：唯一的 SVG 元素，处理 pointer 事件（绘制/选择/拖动），缩放范围 0.5x-3x
- `InspectorPanel.vue`：根据选中结构类型显示对应字段（`elevator` 显示 `code`，`restroom` 显示 `gender`）

6.2 工具栏行为

按钮	行为	快捷键
新增矩形	进入 draw 模式，鼠标按下→拖动→释放后追加 `Structure(source='manual')`	N
删除	删除当前选中项	Delete
撤销	`store.undo()`	Cmd+Z
重做	`store.redo()`	Shift+Cmd+Z
预览	弹窗调用 `FloorPlanSemanticView`（同 Flutter 端的 SVG 渲染）展示效果	—
保存	PUT /structures，成功后 `dirty=false`	Cmd+S
模式切换	PATCH /render-mode；未保存时禁用	—

---

7. SVG 画布渲染规则

7.1 图层顺序（z-index 由低到高）

1. `<rect>` outline 边框 + 浅色填充
2. `<rect>` structures（按类型上色，半透明 0.3）
3. `<line>` windows（橙色 4px）
4. `<rect>` 选中高亮框（虚线边框 + dasharray 4 4）
5. 拖动手柄（8 个控制点，仅选中时显示）

7.2 颜色 Token

颜色统一从 `:root` CSS 变量读取（与项目其他 admin 视图一致）：

结构类型	CSS 变量	默认值
core	`--floor-core`	rgba(150,150,150,0.4)
elevator	`--floor-elevator`	rgba(100,150,255,0.4)
stair	`--floor-stair`	rgba(100,200,150,0.4)
restroom	`--floor-restroom`	rgba(200,150,200,0.4)
column	`--floor-column`	rgba(80,80,80,0.6)

禁止 `style="fill: #abc"` 硬编码。

7.3 交互

- **选中**：单击结构 → store 设置 `selectedIndex`
- **拖动平移**：选中后鼠标按住中心区域 → 修改 `rect.x/y`
- **拖动缩放**：选中后拖动 8 个控制点之一 → 修改对应 `rect.w/h`
- **绘制新矩形**：工具栏「新增」点击 → 设置模式 → 鼠标按下记起点 → 拖动预览 → 释放写入
- **键盘**：方向键移动 1px，Shift+方向键移动 10px

所有操作都通过 store action 记录到历史栈。

---

8. 校验规则（前端预校验）

保存前在 store `save()` 内执行，**与后端 §3.3 保持一致**：

```
function validate(map: FloorMapV2): string | null {
  if (!map.outline) return 'outline 缺失';
  if (map.structures.length > 200) return '结构数量超过 200';
  for (const s of map.structures) {
    if (s.rect.w <= 0 || s.rect.h <= 0) return '矩形尺寸非法';
    if (!isInside(s.rect, map.outline)) return `${s.type} 越出 outline`;
    if (s.type === 'elevator' && !s.code) return '电梯必须填写编号';
    if (s.type === 'restroom' && !s.gender) return '卫生间必须选择性别';
  }
  // IoU 重复检测略
  return null;
}
```

预校验失败 → ElMessage 警告，不发请求。

---

9. 加载与保存交互

9.1 进入页面

```
created → loading=true
       ↓
GET /api/floors/:id/structures (已确认数据)
       ↓
404 FLOOR_MAP_NOT_FOUND → 回退 GET /candidates
       ↓
若仍 409 FLOOR_CANDIDATES_NOT_READY → 显示空状态 + 提示「请先上传 DXF」
       ↓
draft = 深拷贝(候选项 / 已确认值)
loading = false
```

9.2 离开页面

`beforeRouteLeave` 检测 `dirty === true` → ElMessageBox 确认是否丢弃修改。

9.3 保存

```
点击保存 → 前端 validate
       ↓
PUT /api/floors/:id/structures
       ↓
成功: ElMessage success("已保存,版本 vN") + dirty=false
失败: 按错误码映射友好文案,见 §10
```

---

10. 错误码 → 文案映射

```
const ERROR_MESSAGES: Record<string, string> = {
  FLOOR_MAP_INVALID_SCHEMA: '数据格式不符合规范,请检查必填字段',
  FLOOR_MAP_RECT_OUT_OF_BOUNDS: '存在结构越出楼层轮廓,请调整位置',
  FLOOR_MAP_DUPLICATE_STRUCTURE: '存在重复的结构(高度重叠)',
  FLOOR_MAP_TOO_MANY_STRUCTURES: '结构数量超过 200,请精简',
  FLOOR_MAP_TOO_MANY_WINDOWS: '窗户数量超过 200,请精简',
  FLOOR_RENDER_MODE_INVALID: '渲染模式参数非法',
  FLOOR_CANDIDATES_NOT_READY: '该楼层尚未上传 DXF 或抽取尚未完成',
  FLOOR_NOT_FOUND: '楼层不存在',
  FLOOR_MAP_NOT_FOUND: '该楼层尚未做过结构标注',
};
```

不解析 `error.message`，只按 `error.code` 取本地文案（copilot-instructions.md）。

---

11. 性能要点

- SVG 元素总数 ≤ 500（outline 1 + structures 200 + windows 200 + 控制点 8），原生渲染足够；不引入虚拟化
- 历史栈深度上限 50，防止内存膨胀
- 拖动时使用 `requestAnimationFrame` 节流刷新（不每个 mousemove 都触发 store action）；释放鼠标时一次性 commit
- store 不持久化到 localStorage（避免脏数据跨会话）

---

12. 测试

- 单元测试（vitest）： `floorStructuresStore.spec.ts` 覆盖 add/update/remove/undo/redo 与 validate 函数
- 组件测试（@vue/test-utils）： `AnnotatorView.spec.ts` 覆盖加载分支（已确认 / 候选项 / 空状态）
- e2e（playwright）： `e2e/floor-structures.test.ts` 覆盖完整保存流程，使用 fixtures 注入候选项

---

13. 上线前检查清单

- [ ] `assets.write` 权限校验（前后端双层）
- [ ] DXF 未上传场景空状态文案
- [ ] 离开页面确认对话框
- [ ] 撤销/重做快捷键工作
- [ ] 错误码 → 文案表完整覆盖 §10 全部 code
- [ ] 颜色 token 化（无 `#xxx` 硬编码）
- [ ] 单 Vue 文件 ≤ 250 行（超限拆分到 `components/` ）
- [ ] 与 Flutter 端 `FloorPlanSemanticView` 渲染结果视觉一致（手工对比）